Чисельні методи

Лабораторна робота №3

Методи чисельного розв’язку систем нелінійних рівнянь

Виконав:

Студент КН-24

Бевзюк Н.С.

2026

Варіант **1**

1. Мета роботи.

Здобути практичні навички розроблення алгоритмів і програм чисельних методів розв'язання систем нелінійних рівнянь, зокрема методу простої ітерації, а також дослідити процес збіжності шляхом побудови траєкторії пошуку.

2. Завдання.

1) Введення початкових даних:

   Система нелінійних рівнянь $\begin{cases} x = e^x cosy - 1 \\ y = e^x siny + 1 \end{cases}$, початкова точка (–0,9; 1,4), точність $\varepsilon = 10^{-4}$
2) Перевірка умови збіжності:

$$\left\| \frac{\partial \Phi}{\partial x} \right\|_l = \max_i \sum_{j=1}^{n} \left| \frac{\partial \varphi_i(x)}{\partial x_j} \right| < 1$$

3) Обчислення наступного наближення
4) Перевірка умов зупинки:

$$\|x^{(k+1)} - x^{(k)}\| < \varepsilon$$

5) Отримання результатів
6) Траєкторія пошуку графіком та ітераційно
7) Висновки

3. Розрахункові формули чисельного методу.

Математична постановка задачі полягає у знаходженні розв'язку системи n нелінійних рівнянь із n невідомими, яку в загальному вигляді записують як $f_i(x_1, x_2, ..., x_n) = 0$. Для зручності обчислень систему подають у векторно-матричній формі $F(x) = 0$, де x — вектор шуканих змінних, а F(x) — вектор-функція. Оскільки загального аналітичного методу для таких систем не існує, застосовують ітераційні підходи, зокрема метод простої ітерації, який базується на приведенні вихідної системи до вигляду, приданого для знаходження нерухомої точки.

Для застосування методу простої ітерації вихідне рівняння $F(x) = 0$ трансформують у вигляд $x = \Phi(x)$. Цей перехід можна здійснити шляхом виділення змінних xi або за допомогою операторної форми $x = x + AF(x)$, де A — невироджена матриця. Вибір матриці A є критичним для забезпечення збіжності; часто рекомендується використовувати матрицю, обернену до якобіана (матриці Якобі), взятого в початковій точці: $A = -W^{-1}(x^{(0)})$. Матриця

Якобі W(x) складається з частинних похідних функцій системи за кожною зі змінних: $w_{ij} = \partial f_i / \partial x_j$.

Сам алгоритм обчислень має циклічний характер і починається з вибору початкового наближення $x^{(0)}$. На кожному кроці k розраховується наступне наближення за формулою $x^{(k+1)} = \Phi(x^{(k)})$. Процес триває до досягнення заданої точності ε. Критерієм завершення пошуку є умова, при якій норма різниці між двома послідовними наближеннями стає меншою за ε, тобто $||x^{(k+1)} - x^{(k)}|| < \varepsilon$. Після виконання цієї умови останнє отримане значення $x^{(k+1)}$ приймається за достатньо точне рішення системи.

4. Текст програми.

Через специфіку рівнянь в системі (e, sin, cos, …) та своєрідне їх представлення в коді (оскільки використовується python з бібліотекою math), було прийняте рішення захардкодити систему нелінійних рівнянь та залишити для введення лише точку та точність.

ПОЧАТОК КОДУ

```
import tkinter as tk

from tkinter import messagebox, ttk

import math


# функції

def phi1(x, y):

    return math.exp(x) * math.cos(y) - 1

def phi2(x, y):

    return math.exp(x) * math.sin(y) + 1


# збіжність

def check_convergence(x, y):

    e = math.exp(x)


    dphi1_dx = e * math.cos(y)

    dphi1_dy = -e * math.sin(y)

    dphi2_dx = e * math.sin(y)
```

```
    dphi2_dy = e * math.cos(y)

    q1 = abs(dphi1_dx) + abs(dphi1_dy)
    q2 = abs(dphi2_dx) + abs(dphi2_dy)
    return max(q1, q2)

# метод Простої Ітерації
def simple_iteration(x0, y0, eps, max_iter=100):
    x, y = x0, y0
    history = [(x, y)]

    for _ in range(max_iter):
        x_new = phi1(x, y)
        y_new = phi2(x, y)
        history.append((x_new, y_new))

        if max(abs(x_new - x), abs(y_new - y)) < eps:
            return x_new, y_new, history
        x, y = x_new, y_new
    return x, y, history

# графік
def draw_graph(history):
    canvas.delete("all")
    width = 500
    height = 300
    margin = 40

    xs = [p[0] for p in history]
    ys = [p[1] for p in history]
```

```
    min_x, max_x = min(xs), max(xs)
    min_y, max_y = min(ys), max(ys)

    if max_x - min_x == 0:
        max_x += 1
    if max_y - min_y == 0:
        max_y += 1

    def scale_x(x):
        return margin + (x - min_x) / (max_x - min_x) * (width - 2 * margin)
    def scale_y(y):
        return height - (margin + (y - min_y) / (max_y - min_y) * (height - 2 * margin))

    canvas.create_line(margin, height - margin, width - margin, height - margin,
width=2)
    canvas.create_line(margin, margin, margin, height - margin, width=2)
    canvas.create_text(width / 2, height - 10, text="x")
    canvas.create_text(10, height / 2, text="y", angle=90)

    for i in range(len(xs) - 1):
        x1, y1 = scale_x(xs[i]), scale_y(ys[i])
        x2, y2 = scale_x(xs[i + 1]), scale_y(ys[i + 1])
        canvas.create_line(x1, y1, x2, y2, fill="blue", width=2)
        canvas.create_oval(x1 - 3, y1 - 3, x1 + 3, y1 + 3, fill="red")
        canvas.create_text(x1, y1 - 10, text=str(i), font=("Arial", 7))

    x_last, y_last = scale_x(xs[-1]), scale_y(ys[-1])
    canvas.create_oval(x_last - 5, y_last - 5, x_last + 5, y_last + 5, fill="green")

# таблиця ітерацій
```

```
def fill_table(history):
    for row in table.get_children():
        table.delete(row)
    for i, (x, y) in enumerate(history):
        table.insert("", "end", values=(i, f"{x:.5f}", f"{y:.5f}"))

# функція підрахунку
def solve():
    try:
        def parse(e):
            return float(e.get().replace(",", "."))

        x0 = parse(x_entry)
        y0 = parse(y_entry)
        eps = parse(eps_entry)
        q = check_convergence(x0, y0)

        if q < 1:
            convergence_text = (
                f"{q:.4f} < 1 → метод збігається\n"
            )
        else:
            messagebox.showerror(
                "Помилка",
                f"q = {q:.4f} ≥ 1\nМетод може не збігатися"
            )
            return

        x, y, history = simple_iteration(x0, y0, eps)
        result_var.set(
```

```
            convergence_text +
            f"Результат:\n"
            f"x = {x:.5f}\n"
            f"y = {y:.5f}"
        )
        draw_graph(history)
        fill_table(history)

    except Exception as e:
        messagebox.showerror("Помилка", str(e))

# GUI
root = tk.Tk()
root.title("Метод простої ітерації")
frame = tk.Frame(root)
frame.pack(pady=10)

tk.Label(frame, text="Початкова точка").grid(row=0, column=0, columnspan=2)
tk.Label(frame, text="x").grid(row=1, column=0)
tk.Label(frame, text="y").grid(row=1, column=1)

x_entry = tk.Entry(frame, width=10)
y_entry = tk.Entry(frame, width=10)
x_entry.grid(row=2, column=0)
y_entry.grid(row=2, column=1)

x_entry.insert(0, "-0.9")
y_entry.insert(0, "1.4")

tk.Label(root, text="Точність ε").pack()
```

```
eps_entry = tk.Entry(root, width=10)
eps_entry.insert(0, "0.0001")
eps_entry.pack()

tk.Button(root, text="Розв'язати", command=solve).pack(pady=10)
result_var = tk.StringVar()
tk.Label(root, textvariable=result_var, font=("Arial", 11), justify="left").pack()

canvas = tk.Canvas(root, width=500, height=300, bg="white")
canvas.pack(pady=10)
columns = ("k", "x", "y")
table = ttk.Treeview(root, columns=columns, show="headings", height=8)

for c in columns:
    table.heading(c, text=c)
    table.column(c, width=100, anchor="center")

table.pack()
root.mainloop()
```

КІНЕЦЬ КОДУ

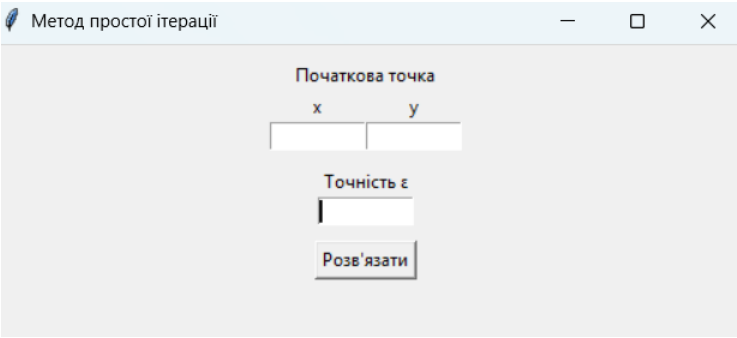
5. Результат роботи програми і траєкторія пошуку.

Дані:

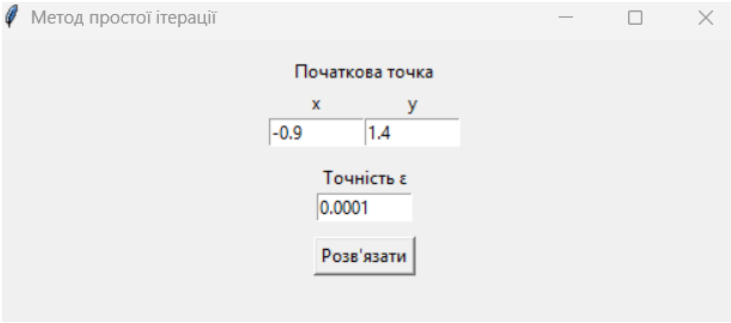
Рівняння - $\begin{cases} x = e^x cosy - 1 \\ y = e^x siny + 1 \end{cases}$

Оцінка похибки ε - 10^{-4}

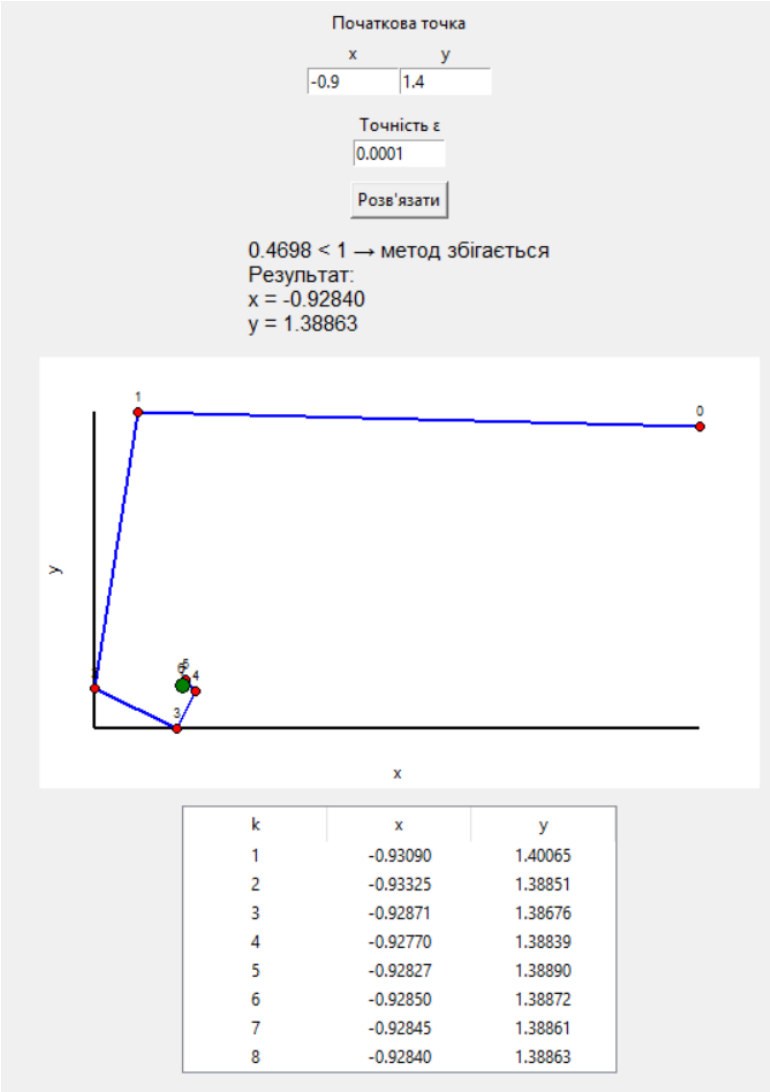
Запуск програми:



Введення початкової точки та наближення:



Натискання на кнопку “ Розв’язати”:

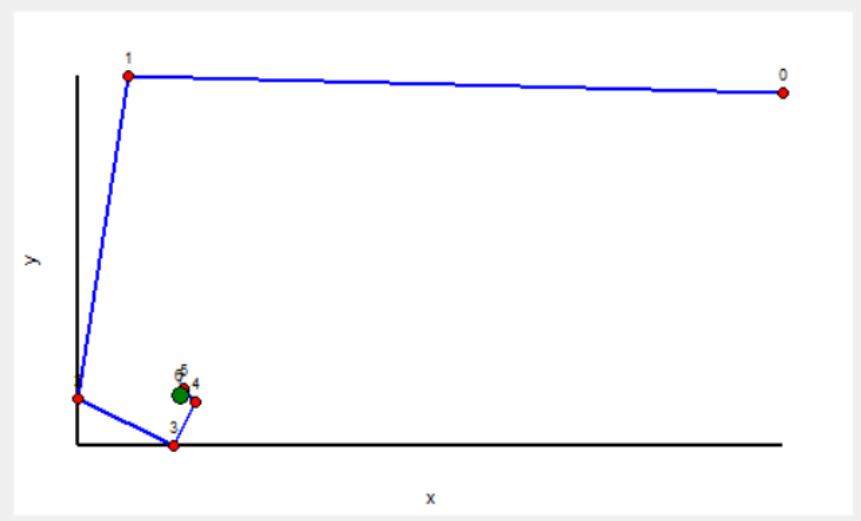


k	x	y
1	-0.93090	1.40065
2	-0.93325	1.38851
3	-0.92871	1.38676
4	-0.92770	1.38839
5	-0.92827	1.38890
6	-0.92850	1.38872
7	-0.92845	1.38861
8	-0.92840	1.38863

Перегляд результату – відповідь:

0.4698 < 1 → метод збігається
Результат:
x = -0.92840
y = 1.38863

Перегляд результату – траєкторії пошуку:



Перегляд результату – таблиця ітерацій:

k	x	y
1	-0.93090	1.40065
2	-0.93325	1.38851
3	-0.92871	1.38676
4	-0.92770	1.38839
5	-0.92827	1.38890
6	-0.92850	1.38872
7	-0.92845	1.38861
8	-0.92840	1.38863

Перевірка результатів:

Отриманні значення x = -0.92840 та y = 1.38863 підставляються у систему нелінійних рівнянь для перевірки:

$$\begin{cases} x = e^x cosy - 1 \\ y = e^x siny + 1 \end{cases} = \begin{cases} -0.92840 \ = e^{-0.92840}\cos(1.38863) - 1 \\ 1.38863 = e^{-0.92840}\sin(1.38863) + 1 \end{cases} =$$

$$= \begin{cases} -0.92840 \ = 0.39518 * 0.18116 - 1 \\ 1.38863 = 0.39518 * 0.98345 + 1 \end{cases} = \begin{cases} -0.92840 \ = 0.07159 - 1 \\ 1.38863 = 0.38864 + 1 \end{cases} =$$

$$= \begin{cases} -0.92840 \ = -0.92841 \\ 1.38863 = 1.38864 \end{cases}.$$

6. Висновки.

У ході виконання лабораторної роботи було реалізовано метод простої ітерації для чисельного розв’язання системи нелінійних рівнянь. Було здійснено перехід до ітераційної форми, обрано початкову точку та задано точність обчислень. Особливу увагу приділено перевірці умови збіжності методу. Було обчислено q, який визначає наявність збіжності ітераційного процесу. У випадку виконання умови $q < 1$ підтверджено, що система є збіжною, а отже, розв’язок існує та може бути знайдений із заданою точністю. Це дозволило обґрунтувати коректність застосування методу для заданої системи.

Отримані результати демонструють ефективність методу простої ітерації для розв’язання нелінійних систем за умови правильної побудови ітераційних функцій та вибору початкового наближення. Побудована програма дозволяє не лише отримати чисельний розв’язок, але й наочно проаналізувати процес збіжності, що є важливим для глибшого розуміння чисельних методів.